ChatGPT

System Architecture Overview

A **voice agent** must process live audio in real time. The user's speech is captured in the browser, streamed to the backend for STT, sent to the LLM for reasoning, then the response is converted to speech and streamed back. A **modular, scalable architecture** can be built with FastAPI (backend), React (frontend), and LangGraph for agent orchestration. Key components:

- **Frontend (React):** Captures microphone audio (e.g. via `MediaRecorder`) in small chunks (~200–300ms [1]) and streams it to the server (using WebRTC or WebSockets). It also plays back TTS audio chunks from the server.
- **Backend (FastAPI):** Exposes endpoints (or WebSocket/RTC streams) for STT, LLM processing, and TTS. It hosts the LangGraph agent loop and manages the voice pipeline.
- **Agent Orchestration (LangGraph):** Implements the conversational "agent loop" (message collection, model calls, tool usage) in a durable, stateful workflow. It takes **static config files** (Identity and Context) as input to shape the agent's behavior.
- **LLM (OpenAI GPT-4o+):** The core reasoning engine. The agent prompt is constructed from the conversation history plus static identity/context.
- **STT (Whisper or Deepgram):** Converts incoming speech to text. This runs as a streaming service (e.g. Deepgram WebSocket API or OpenAI's Whisper API) on the backend.
- **TTS (ElevenLabs):** Converts agent's text reply to audio. ElevenLabs supports *chunked streaming* of raw audio bytes [2] [3].
- **Streaming:** Real-time audio requires low-latency protocols. WebRTC (for media) or WebSockets (for control/data) are used. (See *Realtime Streaming Design* below.)
- **Future Telephony (Twilio/SIP):** When integrated, the VoIP call (Twilio or SIP trunk) passes caller audio into our system and plays TTS back to the caller. For example, Twilio ConversationRelay or Twilio's streaming API can connect phone calls to our FastAPI agent via WebSockets [4] [5].

A step-by-step flow for one conversation turn could be: 1. **User speaks**: Browser captures mic audio, sends audio frames to FastAPI. 2. **STT processing**: FastAPI streams audio to an STT service (Whisper or Deepgram) and receives interim transcripts. 3. **Agent loop**: Transcribed text is added to the conversation state. LangGraph invokes the agent graph, using the static identity/context (see below) plus recent messages to generate a prompt for the LLM. 4. **Model call**: Call OpenAI GPT (e.g. GPT-4o) with a streaming response, receiving reply text. 5. **Tool usage (if any)**: If the agent logic calls any tools (e.g. a knowledge-base or search), that happens here within the LangGraph flow. 6. **TTS processing**: The LLM's textual reply is passed to ElevenLabs for TTS. ElevenLabs returns streamed audio chunks. 7. **Return audio**: FastAPI streams the TTS audio back to the frontend (or out through Twilio) in real time. 8. **Repeat**: The user can speak again; the cycle continues.

This design cleanly separates concerns: **identity/context** (static configs), **agent logic** (LangGraph), **real-time transport** (WebSockets/RTC), and **LLM/TTS/STT** services. It also allows swapping components (e.g. replace ElevenLabs with another TTS) without changing the core flow.

Identity and Context Configuration Files

The agent's **behavior and knowledge** come from two static files:

- **Identity File ("WHO")**: Defines the agent's persona. Fields include:
- `role`: e.g. "Salesman", "SupportAgent".
- `tone`: e.g. "friendly", "professional", "empathic".
- `personality_traits`: e.g. "patient, concise".
- `speaking_style`: e.g. "first-person narrative", "formal".
- `behavior_rules`: e.g. "never speak negatively about the product", "always greet customer by name".

- *Example (YAML)*:

```yaml
role: "Technical Support Agent"
tone: "empathetic"
personality_traits:
  - patient
  - friendly
  - detail-oriented
speaking_style: "conversational"
behavior_rules:
  - "Greet the user by name each time."
  - "Apologize for any inconvenience."
  - "Avoid technical jargon; explain simply."
```

- **Context File ("WHAT")**: Defines knowledge and strategies. Fields include:

- `domain_knowledge`: Background info (e.g. "Product X specs: ...").
- `tools`: List of available tools (e.g. "calculator", "search_api"), and **instructions** on how/when to use them.
- `conversation_strategies`: e.g. "persuasion tips", "objection handling tips".
- `tips`: e.g. "offer discounts on objections".
- `constraints`: e.g. "no sensitive data access", "follow company policy".
- *Example (YAML)*:

```yaml
domain_knowledge:
  product_info: "Our SaaS platform improves team productivity by 30%.
Key features include A, B, C."
  support_docs_url: "https://example.com/support"
tools:
  - name: "web_search"
    description: "Search the company knowledge base."
  - name: "crm_lookup"
    description: "Fetch customer info given an email."
conversation_strategies:
  - "Always confirm the issue before proposing a solution."
  - "If user is frustrated, apologize and empathize first."
```

```
tips:
  - "Highlight any relevant promotions or deals."
constraints:
  - "Do not disclose user personal data."
  - "Stay within compliance guidelines."
```

These files can be JSON or YAML. The system loads them at startup (they are static for now). They are treated as **read-only context** in the LangGraph agent. In LangGraph terms, these values go into the `context` schema for the agent invocation [6]. They do *not* change during the conversation (unless explicitly updated by design), so static context is ideal [7]. By design, the agent "knows" these at runtime as part of its system prompt and logic.

Injecting Identity & Context into LangGraph

In the LangGraph framework, **static context** is the mechanism to pass immutable information (like our identity/context files) into the agent each run [6]. You would define a `context_schema` that includes fields matching your identity and context file keys. For example:

```
class AgentContext(TypedDict):
    role: str
    tone: str
    personality_traits: str
    behavior_rules: str
    domain_knowledge: str
    tools: List[str]
    strategies: str
    # ... etc for all fields needed ...
```

When invoking the agent (e.g. `graph.invoke(initial_state, context=context)`), you provide a dictionary `context` whose values come from the loaded YAML/JSON files. LangGraph makes this data accessible in prompts or tool calls via `runtime.context`. For instance, in a prompt function you might write:

```
def build_prompt(state: MessagesState) -> list[BaseMessage]:
    ctx = state.runtime.context  # static context loaded from files
    system_msg = (
        f"Role: {ctx['role']}, Tone: {ctx['tone']}. "
        f"Personality: {ctx['personality_traits']}. "
        f"Behavior Rules: {ctx['behavior_rules']}. "
        f"Domain knowledge: {ctx['domain_knowledge']}"
    )
    return [SystemMessage(content=system_msg)] + state['messages']
```

This ensures the **identity and context are injected as system instructions**. LangGraph separates `context` (static inputs) from `state` (dynamic messages) [8]. By convention, any value that should not change mid-conversation (e.g. the agent's persona) stays in `context`; dynamic items like

messages or counters stay in `state` [6] [7]. In practice, you might load the YAML at startup, then do something like:

```
identity = load_yaml("identity.yaml")
context_info = load_yaml("context.yaml")
context = {**identity, **context_info}
agent_state = {"messages": []}
graph.invoke(agent_state, context=context)
```

LangGraph will then carry that context through every step of the agent's execution. (See LangChain docs for more on context vs state.) This design cleanly separates **who the agent is** (identity) and **what it knows/does** (context) from the agent's runtime logic.

Project Folder Structure

A clean folder layout helps manage the parts. For example:

```
voice-agent-project/
├── app/
│   ├── main.py                # FastAPI entrypoint
│   ├── api/
│   │   ├── stt_router.py      # WebSocket or HTTP endpoints for STT
│   │   ├── tts_router.py      # Endpoints for TTS streaming
│   │   └── chat_router.py     # Main endpoint for chat (if used)
│   ├── agent/
│   │   ├── graph.py           # LangGraph graph definition (nodes, edges)
│   │   ├── agent_loop.py      # Functions for agent steps (LLM call, tool
usage)
│   │   └── context_schema.py  # TypedDict for LangGraph context/state
│   ├── tools/
│   │   ├── search_tool.py     # Example tool integration
│   │   └── crm_tool.py        # Another example tool
│   └── services/
│       ├── stt_service.py     # Wrappers for Whisper/Deepgram API
│       ├── tts_service.py     # Wrapper for ElevenLabs API
│       └── twilio_service.py  # (Future) Twilio API interactions
├── config/
│   ├── identity.yaml          # Agent identity file
│   └── context.yaml           # Agent context file
├── frontend/
│   ├── public/                # React public assets
│   ├── src/
│   │   ├── App.jsx            # Main React component (handles UI, audio)
│   │   └── services.js        # JS for WebRTC/WebSocket setup
│   └── package.json
├── requirements.txt           # Python dependencies (FastAPI, langgraph,
```

```
elevenlabs, etc.)
└── README.md
```

- **app/main.py** sets up the FastAPI server, includes routers, and initializes the LangGraph agent with loaded config.
- **app/agent** holds the LangGraph definitions (the graph nodes/edges) and any logic for the agent's loop (prompt construction, invoking model, etc.).
- **app/tools** contains any custom tools the agent can call (e.g. a web search or database query).
- **app/services** has code to call external services (STT, TTS, etc.).
- **config/** holds the static YAML/JSON files (identity.yaml, context.yaml).
- **frontend/** is a React app that opens a WebRTC or WebSocket to the backend, captures mic audio, and plays back audio.

This structure enforces a **clean separation of concerns**: config, backend logic, services, frontend.

Agent Loop (LangGraph) and Prompt Design

In code, the **LangGraph agent loop** is defined by building a `StateGraph`. For a simple conversational agent, you might use a **React-style** loop: alternate LLM calls with user inputs. Pseudo-code using LangGraph Python API:

```
from langgraph.graph import StateGraph, START, END
from langgraph.types import MessagesState, BaseMessage, HumanMessage,
AIMessage
from openai import OpenAI  # or use langchain integration

def llm_node(state: MessagesState) -> dict:
    # Construct messages for OpenAI model
    ctx = state.runtime.context  # identity+context
    system_content = (
        f"You are a {ctx['role']} with a {ctx['tone']} tone. "
        f"{ctx['behavior_rules']} Domain info: {ctx['domain_knowledge']} "
    )
    system_msg = {"role": "system", "content": system_content}
    # Combine with conversation history
    prompt = [system_msg] + state["messages"]
    # Call the LLM (streaming)
    ai_response = OpenAI(model="gpt-4o",
streaming=True).chat(messages=prompt)
    # Return as a state update (append to messages)
    return {"messages": state["messages"] +
[AIMessage(content=ai_response.content)]}

# Build graph
graph = StateGraph(MessagesState)
graph.add_node(llm_node, name="llm")
graph.add_edge(START, "llm")
graph.add_edge("llm", END)
graph = graph.compile()
```

```
# Example invoke:
init_state = {"messages": [HumanMessage(content="Hi, I need help with X.")]}
result = graph.invoke(init_state, context=context)
```

This simple graph just runs one LLM call. In practice, you might add **tool nodes**. For example, if tools are available, the agent's prompt template can include instructions like "Use the {tool} to find information." After the LLM generates a response indicating a tool use, the graph can branch to a tool node that executes the tool and then feeds the result back to the LLM in a follow-up call. LangGraph allows adding interruptible subgraphs or tool-call nodes as needed.

Prompt construction uses both static config and dynamic messages. Typically you prepend a `system` message combining identity/context (see above) and then all user + assistant messages. For example:

```
system_msg = (
    f"You are a {ctx['role']} for {ctx['domain_knowledge']}. "
    f"Tune your tone to be {ctx['tone']} and follow these rules:
{ctx['behavior_rules']}. "
    f"Strategy tips: {ctx['conversation_strategies']}. "
)
prompt = [{"role": "system", "content": system_msg}] + state["messages"]
```

This way the LLM "knows" who it is and what it should know.

Tool calling: If a tool call is needed, one pattern is to have the LLM output a structured command (e.g. JSON with a tool name and arguments). LangGraph can check if the LLM's message indicates a tool call, then route that to a tool node. For example:

```
def tool_node(state: MessagesState) -> dict:
    tool_request = state["messages"][-1]
# assume last LLM message requested a tool
    result = call_tool(tool_request.tool_name, tool_request.tool_args)
    # Add tool result as a new assistant message
    return {"messages": state["messages"] + [AIMessage(content=result)]}
```

Edges in the graph connect these appropriately. LangGraph's **state-update semantics** allow you to modify messages, add intermediate "tool result" messages, etc.

Real-Time Streaming: WebSockets vs WebRTC

For live voice, low-latency streaming is critical. Two main approaches:

- **WebSockets**: A full-duplex TCP-based connection. Easy to implement in FastAPI (`WebSocketRoute`). Can send binary audio frames between client and server, and server can stream data back. It is widely supported and simple for arbitrary data. However, WebSocket frames must traverse TCP, so latency can be ~500ms–1s including buffering. For rapid turn-taking, this may be a bit high. Many demo apps use WebSockets for simplicity (e.g. Twilio's

ConversationRelay uses FastAPI + WebSockets [4]). WebSockets also require us to handle audio encoding/decoding manually.

- **WebRTC**: Designed for peer-to-peer low-latency media. It uses UDP and built-in jitter buffering, echo cancellation, etc., achieving sub-200ms latency [9]. It's ideal for real-time audio. In OpenAI's documentation, the **Realtime API** uses WebRTC in the browser by default [10]. For voice agents, the recommended approach is WebRTC on the client side (for microphone capture and playback) with a server-side component (like a TURN server or LiveKit) bridging to the backend. WebRTC is more complex to set up (needs STUN/TURN servers, SDP negotiation) but it offloads real-time media.

Recommendation: For a first implementation, WebSockets suffice (and align with FastAPI's built-in support). You would stream audio chunks from React to FastAPI, and stream audio bytes back as they arrive. This is simpler to demo (no separate signaling). If ultra-low latency is needed (<500ms) and you're comfortable with the complexity, WebRTC (perhaps via a library like aiortc or a managed service like LiveKit) is better [9] [11]. WebRTC handles live audio natively and can reduce round-trip delays. For example, OpenAI suggests using WebRTC for browser interactions with their realtime models [10].

In practice, a hybrid can be used: Use WebRTC (or getUserMedia→MediaRecorder) in the browser to capture audio, then send it over a WebRTC data channel or via a WebSocket in small chunks. (Since we're not building a conferencing app, a WebSocket often suffices for sending PCM/WAV frames.) For output, you can stream raw audio bytes over WebSocket and play them with the Web Audio API, or have the client pull streaming audio from an endpoint.

Voice Pipeline (STT → LLM → TTS)

STT (Speech-to-Text): Ingest streaming audio and transcribe. Two options:

- **OpenAI Whisper API:** Currently requires sending chunks via HTTP; not truly streaming. (The *Realtime API* may eventually include Whisper streaming [12], but as of 2026 it's not real-time.) A practical approach is to send small audio snippets (e.g. 250ms or 1sec) to Whisper and get transcripts. This introduces some latency.
- **Deepgram:** Provides a true streaming WebSocket API. You open a WebSocket (`wss://api.deepgram.com/v1/listen?model=…&language=en`) and send raw audio frames. Deepgram returns partial transcripts in JSON for each chunk [13]. This matches the real-time loop: "As the user speaks, their voice is captured in small audio chunks (~250ms) and streamed to the backend. The backend (Deepgram) responds continuously with transcripts" [1]. Use Deepgram's Python SDK or WebSocket to integrate. The pipeline is: React → WebSocket (FastAPI) → Deepgram → text.

LLM (Language Model): Take the transcript text, append it to the conversation history, build the prompt (including identity/context), and call the OpenAI LLM. For real-time feel, use streaming API if available so you can start TTS as the text arrives (but GPT-4o's streaming is text streaming, not audio). If using OpenAI's *Realtime* endpoint (speech-to-speech models like `whisper-3o` and `whisper-4o` if they exist), you could connect via WebRTC directly, but since we use GPT-4o (text-based), we do text in/out.

TTS (Text-to-Speech): Once the LLM yields a response text, send it to ElevenLabs. ElevenLabs offers a streaming endpoint: `elevenlabs.text_to_speech.stream(...)` returns chunks of raw audio bytes [2] [3]. You can iterate over the stream and pipe audio chunks to the client. For example,

ElevenLabs' Python SDK returns an iterator of MP3 bytes which you can write to a `BytesIO` or directly to the HTTP/WebSocket response [14] [2]. Because ElevenLabs buffers minimally and sends chunks as available, you can push audio quickly to the user. Use a low-latency model (e.g. `eleven_flash_v2_5`) and small output format (e.g. low-sample-rate) for faster streaming [15].

Putting it together: The backend pipeline is:

```
flowchart LR
  A[Browser (Microphone)] -->|Audio chunks| B[FastAPI (WebSocket/RTC)]
  B --> C[Speech-to-Text (Deepgram/Whisper)]
  C --> D[LangGraph Agent (messages, prompt)]
  D --> E[LLM (OpenAI GPT-4o)]
  E --> F[LangGraph (possibly tools usage)]
  F --> G[Text-to-Speech (ElevenLabs)]
  G -->|Audio chunks| B
```

This ensures each piece is streamable and can run in parallel where possible (e.g. while TTS is playing earlier part, you could process next user audio if multi-threaded).

ElevenLabs Streaming Integration

ElevenLabs provides real-time streaming for TTS. According to their docs: *"The ElevenLabs API supports real-time audio streaming for select endpoints, returning raw audio bytes (e.g. MP3 data) directly over HTTP using chunked transfer encoding"* [2]. In Python, you do:

```python
from elevenlabs import ElevenLabsClient
client = ElevenLabsClient(api_key="…")
audio_stream = client.text_to_speech.stream(
    text="Hello world",
    voice_id="v1_voice_id",
    model_id="eleven_multilingual_v2"
)
for chunk in audio_stream:  # yields bytes
    if chunk:
        websocket.send_bytes(chunk)  # or write to response
```

Because the response is chunked, you can forward each chunk immediately. ElevenLabs also offers WebSocket support, but the HTTP chunking works well with FastAPI by using `StreamingResponse`.

For low-latency, choose the "flash" or "multilingual" models and a moderate audio format (e.g. `ulaw_8000` for telephony, or `mp3_22050_32` for clarity) [15]. In the architecture, after the LangGraph node produces `AIMessage`, call ElevenLabs and stream its output to the frontend socket as it arrives. This lets the user hear the agent speak nearly in sync.

Low-Latency Best Practices

To keep end-to-end latency under ~1–2 seconds:

- **Chunking:** Use small audio chunks (200–300ms) for STT. This minimizes wait to send and process each piece [1].
- **Streaming APIs:** Use streaming modes at every stage (Deepgram WebSockets, LLM streaming if available, ElevenLabs streaming) so processing overlaps. For example, start TTS as soon as part of the text is generated.
- **Concurrent Pipeline:** Run STT, LLM, TTS pipelines in parallel as much as possible. While the user speaks, process transcripts and prepare replies in the background.
- **Edge Deployment:** Host FastAPI close to users (e.g. on a cloud region) and use a CDN or STUN/TURN for WebRTC to reduce network delay. Use `uvicorn --workers` to handle concurrency.
- **Speech Activity Detection (VAD):** Don't send silence to the model. Detect when user stops speaking (via audio energy) before sending a final "end of turn" to LLM. This avoids wasted tokens and lowers cost [16].
- **Efficient Models:** For some parts of the conversation, consider using a smaller/cheaper model (GPT-3.5 for general chit-chat, GPT-4o for complex queries). This reduces token count and may increase throughput.
- **Batching WebSocket Frames:** On WebSocket transport, do not send every millisecond; buffer a short window (e.g. 100ms audio) then send to avoid thrashing.
- **Client-Side Buffer:** Keep a small audio buffer (a few hundred ms) on the client so jitter is absorbed, preventing stutter.
- **Profiling:** Monitor where delays occur (STT latency, LLM response time, etc.) and optimize those. Use async I/O everywhere.

With these, a <1s round-trip is feasible for short queries, especially using WebRTC for audio. If using WebSockets, expect ~500ms–1s depending on model speed and network.

Phone Call Integration (Twilio/SIP)

For telephony, you have two main paths:

1. **Twilio Voice + ConversationRelay**: Twilio offers a product called ConversationRelay that bridges voice calls to an LLM via WebSocket [4]. It essentially works like this: Twilio streams the call's audio into your FastAPI endpoint (via WebSocket), and you stream responses back to Twilio, which plays them to the caller. Using ConversationRelay, you mainly implement a FastAPI WebSocket server that receives "media" and sends back TTS. The Twilio docs show an example (using FastAPI/Express) where an incoming call hits your `/incoming` webhook, TwiML `<Stream>` connects to a WebSocket on your server, and your server processes audio and sends back audio packets to Twilio's client [17]. The TenLabs Twilio guide (JavaScript example) demonstrates using `twiml.connect().stream()` to a WS endpoint [18]. In Python, one would similarly implement an async WebSocket handler and use `VoiceResponse` to connect.

2. **SIP Trunk / WebRTC**: The OpenAI Realtime API supports SIP for VoIP. Twilio can act as a SIP provider. You could also use Asterisk or FreeSWITCH. The idea: ring the Twilio number, Twilio SIPs to your backend URL (or vice versa), and your app handles the SIP session, bridging into the same STT/LLM/TTS pipeline. This is more complex than using ConversationRelay.

Design for Twilio: In practice, treat phone audio like any stream. For example, using ConversationRelay: set up a TwiML webhook so that when a call arrives, Twilio will initiate a WebSocket connection to your `/stream` endpoint. On that WS, you'll get JSON events (`{"event":"media", "media": {"payload": ...}}`) with base64 audio, and you respond by sending back audio frames base64-encoded. Your backend code will call STT on incoming frames, send text to LangGraph, then send TTS frames back via Twilio's protocol. This is effectively the same pipeline, just with a different transport. The Deepgram Twilio guide outlines a similar path [19] (Deepgram STT and TTS with Twilio).

Either way, **decouple phone logic from core agent logic**: use an adapter or service that converts Twilio/SIP streams into the same WebSocket stream interface that your React client would use. That way the LangGraph core doesn't care if input comes from browser or phone – it's just audio/text.

Cost Optimization (Beyond Free Tiers)

To keep costs manageable:

- **Model Selection:** Use GPT-4o only when needed. For simpler responses or as initial engagement, try GPT-3.5 (if allowed). Possibly "tiered" approach: cheap model first, fall back to GPT-4o if the user insists or complex queries arise.
- **Silence Skipping (VAD):** As noted, do not send silent chunks to the LLM or STT. This saves both API usage and compute [16].
- **Token Management:** Keep prompts concise. Since the conversation can grow, implement a summarization or memory mechanism: e.g. trim older messages, use embeddings for long history. (LangGraph supports memory via the Store, if needed.)
- **Fixed Context Injection:** Since identity/context are static, include them once in the system prompt rather than repeatedly each turn. (Though often a system message is repeated, you can store parts in system memory if needed.)
- **TTS Settings:** ElevenLabs free tier has limited characters. Beyond that, compare alternatives (e.g. Amazon Polly, Google Cloud TTS, or smaller models like Coqui TTS). Some services charge per character or per minute; use shorter responses or silence.
- **STT Cost:** Whisper API charges per minute of audio; Deepgram charges per second. For normal dialogs, the voice minutes might be limited but watch usage. If usage is high, consider a tiered approach (maybe local Whisper for silence detection, and only use paid STT on speech).
- **Infrastructure:** Use autoscaling for the FastAPI backend so you pay for compute only when needed. Use spot instances or serverless where possible. Cache results of tools (if queries repeat).
- **Monitoring:** Use LangSmith or custom logging to monitor token usage and latency, and adjust accordingly.

Overall, aggressive VAD, careful model choice, and efficient streaming can keep costs low while maintaining performance.

GitHub/LinkedIn Demo Tips and Viral Features

To make the project stand out:

- **Live Demo:** Create a short video/GIF showing a real-time conversation: user speaks naturally, the agent replies in a human-like voice (with expressive tone if possible). Emphasize the low latency and personality.

- **Identity Switching:** Show an example where the same codebase loads a different identity file. For instance, switch between “Sales Agent” and “Customer Support”. Maybe a UI toggle that reloads the identity YAML and restarts the agent with a new persona. Visually, you could show side-by-side transcripts: the same question asked to two agents, with one responding cheerfully and the other formally.
- **Context Switching:** Similarly, switch the context file from a “sales” domain to a “tech support” domain. For example, ask product questions: in sales mode the agent pitches features, in support mode it troubleshoots issues. A video could illustrate the agent responding differently depending on context YAML.
- **Standout Features (Viral Ideas):**
- **Personality Voices:** Beyond tone, try **emotional TTS**: since ElevenLabs supports style/similarity settings, configure the voice to match personality (e.g. an enthusiastic energetic voice for a sales persona). Demonstrate an “anger mode” or “excited mode” using style parameters – this could be eye-catching.
- **Mirror Channel (Whisper-Thousand-Tongues):** A fun demo where the agent translates the conversation into multiple languages or accents on the fly. For example, user speaks English, agent answers in English but with a British or cartoon accent using ElevenLabs voice cloning.
- **Augmented Memory:** Show switching memory on/off. E.g. agent first learns user name or preferences (via “context switching” mid-chat) and then uses them later, highlighting the LangGraph state/store capabilities.

- **Knowledge Tools:** A tool that shows the agent fetching live data (e.g. “Check product stock” or “Search latest news”). Show this in a GitHub-friendly way by making the browser show an external info box (like a mini-knowledge panel) during the conversation.

- **Visualizations:** Include architecture diagrams (flowcharts) in the README. For example, a Sankey diagram of audio->text->LLM->text->audio might illustrate the flow. Use a sequence diagram to show the interactions (user sends audio, API calls, etc.). Tools like Mermaid (in markdown) can make these diagrams.

- **Repository:** A clear README with usage examples, config file templates, and dev setup will attract viewers. Possibly include a Dockerfile to make it easy to run.
- **LinkedIn Articles:** Write a brief post explaining the “magic” under the hood (in simple terms) and link to a demo. Highlight unique angles (voice-first LLM agent with personality).
- **Open-Source Friendly:** If possible, separate paid secrets from code, use free tiers, and encourage others to clone and customize the identity/context for their own demos.

By emphasizing the **identity vs context** separation and showing dynamic switching in demos, you make the project concept memorable. For instance, a quick video comparing how “Alice the Sales Rep” vs “Bob the Support Guy” answer the same prompt will demonstrate the value of the static config files.

Summary

This design uses **FastAPI + LangGraph + React + OpenAI + ElevenLabs/Deepgram** to build a real-time, voice-driven AI agent. The static **identity** and **context** files define “who” the agent is and “what” it knows. These are loaded into LangGraph’s context, form the system prompt, and guide the agent’s behavior [6] [7]. The agent loop in LangGraph handles streaming LLM calls and any tool executions. Audio streams from client→server (via WebRTC/WebSocket) and TTS streams back using ElevenLabs’s chunked API [2]. Low-latency is achieved by streaming at every stage and optimizing each component. Future phone support is planned via Twilio or SIP bridges (Twilio’s ConversationRelay and ElevenLabs Twilio tutorial [4] [18]). Cost is managed by selective model use and silence skipping [16]. With modular

code and clear configs, the project will be easy to deploy, extend, and showcase. All parts are implementation-ready and can be handed off for coding.

Sources: LangGraph and LangChain docs [8] [6] ; ElevenLabs API docs [2] [3] ; real-time streaming best practices [1] [9] ; Twilio voice agent guides [19] [4] ; and Twilio/ElevenLabs integration examples [18] [11] . These informed the design choices and recommendations above.

---

[1] [13] Build a Real-Time Transcription App with React and Deepgram
https://deepgram.com/learn/build-a-real-time-transcription-app-with-react-and-deepgram

[2] [3] Streaming | ElevenLabs Documentation
https://elevenlabs.io/docs/api-reference/streaming

[4] Build a Real-Time Voice AI Assistant with Twilio's ConversationRelay, LiteLLM, and Python | Twilio
https://www.twilio.com/en-us/blog/developers/tutorials/product/voice-ai-assistant-conversationrelay-litellm-python

[5] [10] [12] Realtime API | OpenAI API
https://developers.openai.com/api/docs/guides/realtime

[6] [7] [8] Is Context for static information or for langgraph assistant configurations? - OSS Product Help - LangChain Forum
https://forum.langchain.com/t/is-context-for-static-information-or-for-langgraph-assistant-configurations/2680

[9] WebSocket vs WebRTC for Real-Time Audio Communication | by Inseong Han | Medium
https://medium.com/@inssa1102/websocket-vs-webrtc-for-real-time-audio-communication-d3b05edb5f41

[11] [16] WebRTC vs WebSocket for OpenAI Realtime Voice API Integration: Necessary or Overkill? : r/WebRTC
https://www.reddit.com/r/WebRTC/comments/1g7hqmr/webrtc_vs_websocket_for_openai_realtime_voice_api/

[14] [15] Streaming text to speech | ElevenLabs Documentation
https://elevenlabs.io/docs/eleven-api/guides/how-to/text-to-speech/streaming

[17] [18] Sending generated audio through Twilio | ElevenLabs Documentation
https://elevenlabs.io/docs/eleven-api/guides/how-to/text-to-speech/twilio

[19] Build a Voice Agent with Twilio & OpenAI & Deepgram | Deepgram's Docs
https://developers.deepgram.com/docs/build-voice-agent-with-twilio-deepgram-openai